Day 13 - Practical: Project Structure, npm Scripts, Linting, Git Workflow

Day 13 — Practical: Project Structure, npm Scripts, Linting, Git Workflow

Objectives

- Learn the conventional way a real TypeScript/JavaScript project is organized
- Understand `package.json` in depth, and npm scripts
- Understand `tsconfig.json`'s most important settings
- Set up ESLint and Prettier — the standard linting/formatting tools for this ecosystem
- Practice a real git workflow, applied to a TypeScript project

A conventional project layout

```
my-project/
■■■ node_modules/            # installed packages (Day 6) -- NEVER commit this to git
■■■ .gitignore
■■■ package.json
■■■ package-lock.json
■■■ tsconfig.json
■■■ .eslintrc.json (or eslint.config.js)
■■■ .prettierrc
■■■ README.md
■■■ src/
■   ■■■ index.ts
■   ■■■ utils.ts
■■■ tests/
    ■■■ utils.test.ts
```

This should look familiar if you've seen the Python track's Day 13 "src layout" — the same underlying reasoning applies: keeping your actual source code inside `src/`, separate from configuration files sitting at the project root, keeps a growing project organized and matches what you'll find in essentially every real TypeScript project you open at a job.

`package.json` in depth

You've used `npm init` and `npm install` already (Day 6). Here's a more complete `package.json`:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "test": "jest",
```

```
    "lint": "eslint src"
  },
  "dependencies": {
    "express": "^4.18.0"
  },
  "devDependencies": {
    "typescript": "^5.0.0",
    "jest": "^29.0.0",
    "eslint": "^9.0.0"
  }
}
```

`dependencies` are packages your *finished, running* program needs (like a web framework such as `express`); `devDependencies` are packages needed only for *developing* it (the TypeScript compiler, Jest, ESLint) — this exact distinction mirrors the Python track's Day 13 `dependencies`/`optional-dependencies.dev` split, if you've seen it, and the reasoning is identical: someone just *running* your finished project doesn't need your testing or linting tools installed alongside it.

npm scripts — named shortcuts for common commands

The `"scripts"` section defines shortcuts you run with `npm run <name>` (a couple of very common ones, `start` and `test`, can be run without the word `run`: just `npm start` / `npm test`):

```
npm run build     # runs "tsc" -- compiles your TypeScript
npm start            # runs "node dist/index.js" -- runs the compiled output
npm test               # runs "jest" -- runs your test suite
npm run lint              # runs "eslint src" -- checks your code for problems
```

This is genuinely important, practical convention: instead of every developer on a team needing to remember (or look up) the exact underlying command for building/testing/linting a specific project, everyone just runs `npm run build`, `npm test`, and so on — the *specific* underlying tool and its exact flags are hidden behind these consistent, memorable names, defined once in `package.json`.

`tsconfig.json`'s most important settings

You generated one on Day 11/12 with `npx tsc --init`. A few settings worth understanding, since you'll see them in every real TypeScript project:

```
{
  "compilerOptions": {
    "target": "ES2020",         // which JavaScript version to compile DOWN to (older = more compati
    "module": "CommonJS",          // which module system to compile to (CommonJS for Node, "ESNext"
    "strict": true,                   // turns on ALL of TypeScript's strictest checks -- always en
    "outDir": "./dist",                  // where compiled .js files should be written, keeping th
    "rootDir": "./src"                      // where your TypeScript source files live
  }
}
```

`"strict": true` **deserves special attention: always turn this on for any new project.** Among other things, it enables `strictNullChecks`, which forces you to explicitly handle the possibility of `null`/`undefined` wherever they might occur (recall Day 1's `null`/`undefined` distinction) — without it, TypeScript is considerably more permissive and misses a meaningful category of the exact bugs it exists to catch in the first place. Every professional TypeScript project you'll encounter enables `strict` mode; there's no good reason to leave it off for new code.

ESLint and Prettier — the standard linting/formatting tools

Just like the Python track's `ruff`/`black` (if you've seen that lesson), the JavaScript/TypeScript ecosystem has its own equivalent, widely-adopted pair of tools:

- **ESLint** — a **linter**: scans your code for likely bugs and bad patterns (an unused variable, a suspicious comparison, a Promise you forgot to `await`) without actually running it.
- **Prettier** — an **auto-formatter**: rewrites your code's formatting (indentation, quote style, line length) to one single, consistent style, ending any team debate about exactly how code should look, since the tool simply decides for everyone.

```
npm install --save-dev eslint prettier
npx eslint --init         # walks you through creating a starter ESLint config
npx eslint src               # check your code for problems
npx prettier --write src        # reformat your code automatically
```

Exactly like the Python track's equivalent tools, these are typically wired into a real project's **CI** (an automated system that runs checks every time code is pushed) and often into a "pre-commit hook" that runs automatically before a commit is even allowed to complete. Running them yourself, locally, before pushing, saves the frustrating cycle of pushing code and having CI reject it over a purely cosmetic issue you could have caught yourself in seconds.

The real git workflow, applied to a TypeScript project

This section is identical in spirit to the Python track's Day 13 (if you've done it) — the underlying discipline of version control doesn't change between languages, only which files you're tracking:

```
git init
git add .
git commit -m "Initial commit: project scaffold"

# ... after making changes ...
git status
git diff
git add src/
git commit -m "Add user validation logic"

git checkout -b feature/add-export-command
# ... work, commit ...
git push -u origin feature/add-export-command
```

Commit messages should explain *why*, not restate *what* — `git diff` already shows the "what" in full detail. Avoid committing directly to `main`** on any team project; work through a branch and a reviewed pull request instead, exactly as covered in the Python track's Day 13, if you've seen it.

`.gitignore` for a TypeScript/JavaScript project

```
node_modules/
dist/
*.log
.env
coverage/
```

`node_modules/` (Day 6) is large and entirely recreated by `npm install` from `package.json`/`package-lock.json` — never commit it. `dist/` (or wherever your compiled JavaScript output goes) is a derived artifact from your TypeScript source, exactly like the Python track's `__pycache__/` — regenerated automatically, never hand-edited, so it doesn't belong in git either. `coverage/` is generated by test tools reporting how much of your code your tests actually exercise — also derived, also excluded.

Exercises

This is a hands-on, do-it-for-real day rather than an auto-graded one. Working inside this `day13-practical` folder:

1. Create a `src/greeter.ts` exporting a function `greet(name: string): string` returning `` `Hello, ${name}!` ``.
2. Create a `tests/greeter.test.ts` with a Jest test for `greet` (run it with `npx jest day13-practical` from the track root).
3. Write a `package.json` for this small folder (or reuse the shared root one) with `"build"`, `"test"` scripts.
4. Write a `.gitignore` covering `node_modules/`, `dist/`, `*.log`, `.env`, `coverage/`.
5. If you have git installed: `git init` here, then make two separate commits — one for the source, one for the tests plus `.gitignore` — each with a message explaining *why*.

A worked reference is in `solution/` if you get stuck on the mechanics — but run the actual git commands yourself.